

Benchmarking Methodology

Deep Dive into SGLang — Chapter 33

Yin Yang

Foundation Books Project

Where this chapter sits

This is the chapter that turns “it got faster” into “here is the row you can reproduce.”

- A benchmark number is a **score plus a reproducibility claim** about one workload, one run setup, one source revision.
- Everything Volume I measured — TTFT, ITL, TPOT, throughput — is what a benchmark protocol now has to *pin down* before recording.
- The distinctive risk is not the arithmetic; it is letting the **measured population** drift between two runs.

Goal: by the end you can look at a speedup and say what workload, what window, what reducer, and what source state produced it.

The contract

The surfaces

What to carry forward

The contract

A benchmark number is a protocol, not a score

An engineer reports a scheduler variant raises output throughput 12%.

- Was it the scheduler? A shorter trace? A warmer cache? A different streaming mode? A reducer that changed under the run?
- The number alone cannot answer — so the number alone is not evidence.

A reproducible pipeline preserves the **run contract**: model, server config, hardware, traffic, warmup, measurement window, metric formulas, and retained artifacts.

The comparison $\Delta M = M(\text{variant}) - M(\text{baseline})$ is evidence only when everything except the variable under test stayed fixed.

The running example: one fixed trace

Field	Value
Model	google/gemma-4-E4B-it
Hardware	one-GPU H200-class reservation
Warmup	10 requests (excluded)
Main trace	100 requests, fixed prompt / output lengths
Compared	baseline server vs. one scheduler variant

No performance result is implied — the trace is the *protocol target*. A measured row only appears when an artifact supplies it.

The surfaces

Five benchmark surfaces answer different questions

Surface	Measures	Main source
Online serving	TTFT, ITL, TPOT, E2E, throughput under arrivals	<code>bench_serving</code>
Offline throughput	batch generation latency, token throughput	<code>bench_offline_throughput</code>
One-batch / kernel	prefill/decode micro-costs	<code>test_utils.py</code>
Registered perf	repeatable regression thresholds	<code>test/registered/perf/</code>
Project harnesses	scenario tasks (structured, MM, cache)	<code>benchmark/</code>

The same metric name can sit on different evidence layers — so the surface is part of the claim.

What to carry forward

The four buckets every reviewer checks

Workload model, tokenizer, request population, arrival process, routing keys, LoRA distribution, warmup.

Runtime hardware, server flags, streaming mode, benchmark layer.

Metric reducer convention, retokenization, percentile rule, success criterion.

Artifact command, source revision, environment, aggregate row, optional detail JSONL.

Then confirm baseline and variant differ only in the intended variable. If any bucket fails, ΔM is not evidence.

What the sections build

1. **Pipeline & protocol** — the run contract and measurement window.
2. **Workload & phases** — dataset rows, arrivals, warmup, reduction.
3. **Metrics & aggregation** — the reducer, percentiles, streaming care.
4. **Comparability** — population invariants that keep ΔM honest.
5. **Surface choice** — match the surface to the claim, mean vs. tail.
6. **Artifact & CI** — what travels with the number; regression wrappers.
7. **Promotion & evidence** — when a row becomes reader-facing.

Every mechanism has a home in real SGLang source — that link is the spine.

A benchmark result is a protocol plus a metric. Pin the population; the number follows.